

Flipbox Builder

PROJECT SUMMARY

Instructions: Complete this template and submit it alongside your repository link. Answers can be brief - a few sentences per section is usually enough. Please stop after approximately 4-6 focused hours; unfinished work documented below will not count against you merely because you respected the timebox. Write what is useful for a reviewer to understand your work.

Candidate name	Colette Pitamba	Date submitted	10/02/26
Repository link	https://github.com/Colette-Pitamba/flipbox-builder		
Focused time spent	5 hours including investigation, setup, + documentation		

Setup and running the project

Confirm your README is complete or note anything a reviewer should know before installing and running the app that isn't already in the README.

The README covers install and run

Note:

- `npm audit` reported 0 vulnerabilities at the time of submission.
- Two packages were installed, `@tiptap/core` and `@tiptap/pm` (described below)

Key decisions and rationale

Describe your component architecture, data model, persistence approach, and accessibility approach, and why you chose them. Note any supporting packages you added beyond the starter and explain why.

Component architecture

1. `App.vue` owns the single flipbox object and passes it to both panels.
2. `FlipboxBuilder.vue` holds the two editors and the persistence logic.
3. `RichTextEditor.vue` reusable TipTap editor used twice (front and back).
4. I kept the initial structure on startup and added one folder `/sr/editor` holds the TipTap configuration shared by the editor and the preview.
 - a. `extensions.js` defines the allowed formatting
 - b. `sanitize.js` cleans the HTML before the preview renders it.

Data model

- In the app: `{ front: string, back: string }`, where each value is HTML from TipTap's `getHTML()`. I kept HTML because the starter's `v-model` wiring already uses it and the preview needs HTML to render.
- In storage: `{ version: 1, front, back, savedAt }`.
 - - `version` lets a future change to the shape be migrated or safely ignored.
 - - On load, the data is checked: right version, and string `front` and `back`. Anything else is ignored with a console warning instead of breaking the page.

Persistence approach

- Autosave with a 400 ms debounce. LocalStorage isn't written on every keystroke.
- Saves are skipped when nothing has changed since the last save.
- The normal status line ("Saving..." / "All changes saved") deliberately has no `role` or `aria-live`. Announcing it on every pause in typing would be noise.

Accessibility

- Toolbar
 - Buttons are visible text labels.
 - Toggle buttons expose ``aria-pressed``. The active style uses weight and border as well as color.
 - Each toolbar is named "Front formatting" or "Back formatting."
 - Undo and Redo use ``aria-disabled`` rather than ``disabled``, so they stay focusable.
- Editors
 - One Tab stop, with arrow keys, Home and End moving between buttons.
- Flip state
 - Flip state shown without relying on animation
 - A "Showing front (1 of 2)" line sits under the preview card.
 - The flip button's label changes ("Flip to back" / "Flip to front").

Supporting Packages

- `@tiptap/core` and `@tiptap/pm`. These were already installed as TipTap dependencies. I listed them in package.json because `sanitize.js` imports them directly.

Trade-offs, unfinished work, and other notes

Note anything you chose not to implement (required or optional), anything you would do differently with more time, any known gaps or limitations, and anything else you'd like a reviewer to know.

Chose not to implement

- No automated tests. This was a time box choice.
- Flipping is button only. Clicking the card itself doesn't flip it.
- No "clear" or "reset button"

Known gaps and limitations

- No storage event syncing so if open in multiple tabs, only the last save wins.
- Card faces are labeled regions so Voiceover asks the user to "enter" the region before reading the content.

What I would do differently with more time

- Add unit tests
- Add multi-tab syncing
- Add styling
- Test with mobile screen readers

Testing and verification

Briefly describe what you tested, how you tested it, any problem your testing caught, and the most important behavior or risk that remains untested. Include automated and manual checks where relevant.

Tests performed

- I edited both sides with every format and confirmed the preview matched while typing, including while the card showed the back.
- Refreshed after saving.
- Refreshed immediately after typing.
- Keyboard only (Chrome).
 - Tab order, and arrow-key navigation within the toolbars.
 - Flipping with Enter and Space.
- VoiceOver (Safari, macOS).
 - Toolbar and button names pressed
 - Editor labels and the flip announcement.
 - Only the visible side is readable.

Use of AI-assisted tools

Note which AI-assisted tools you used, what you used them for, how you verified their output, and anything you rejected or had to correct. If you did not use an AI-assisted tool, state that

clearly. If you used any custom instructions, agents, or skills to guide these tools, list them here and confirm the corresponding files are included in your repository.

Tool used: Claude app

What I used it for

- Reviewing the starter's `package-lock.json` for anything suspicious.
- Generating the implementation of the three starter components plus
- Debugging integration errors.
- Drafting the README and this summary.

How I verified its output

- One file at a time. I integrated the files one by one, ran the app after each, and read each file before moving on, so problems could be traced to a single change.

No custom instructions, agents, or skills were used.